

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

1. Case Study KBM	4
1.1. Bối cảnh bài toán	4
1.1.1. Sáu vấn đề cần giải quyết	4
1.1.2. Nhóm người dùng	5
1.1.3. Ràng buộc thực tế	5
1.2. Bốn trụ cột nghiệp vụ	5
1.2.1. SQL Lab — Chăm SQL tự động	6
1.2.2. Network Lab — Chăm lập trình mạng	6
1.2.3. Course Management — Quản lý đào tạo	7
1.2.4. DevOps Lab — Thực hành DevOps/Kubernetes	7
1.2.5. Thành phần hỗ trợ	7
1.3. Quá trình tiến hóa — Từ Monolith đến Microservices	7
1.3.1. Era 1: Distributed Monolith (2016–2022)	7
1.3.2. Era 2: SQL Judge — Sáng tạo từ ràng buộc (2023)	8
1.3.3. Era 3: Thiết kế Microservices (2024)	8
1.3.4. Era 4: Migration và Mở rộng (2025)	8
1.3.5. Era 5: Polyglot — DevOps Lab (2025–2026)	9
1.4. Kiến trúc hiện tại	9
1.4.1. Bounded Contexts	9
1.4.2. Tech Stack	10
1.4.3. Danh mục dịch vụ	11
1.4.4. Tích hợp hệ thống bên ngoài	13
1.4.5. Mô hình triển khai	13
1.5. Trade-offs và Gaps — Bài học từ thực tế	13
1.5.1. Critical — 8 mục (tính đúng đắn / bảo mật)	14
1.5.2. Major — 11 mục (khả năng mở rộng / bảo trì)	14
1.5.3. Minor — 6 mục (cải thiện chất lượng)	15
1.6. Sự cố thực tế — Bài học vàng	15
1.6.1. Sự cố 1: Duplicate Submissions trong kỳ thi	15
1.6.2. Sự cố 2: Dữ liệu không nhất quán giữa các bảng	15
1.6.3. Sự cố 3: Multi-instance Judge — Định tuyến sai	16
1.6.4. Sự cố 4: Identity Merge — Khủng hoảng danh tính	16
1.7. Ảnh xạ Case Study ↔ Chương sách	16
1.8. Contributors	17

1.

CHƯƠNG 1.

Case Study KBM

Phụ lục này trình bày đầy đủ về **KBM** — case study xuyên suốt cuốn sách. Từ bối cảnh bài toán ban đầu, qua quá trình tiến hóa kiến trúc kéo dài gần một thập kỷ, đến kiến trúc microservices hiện tại cùng những trade-off và sự cố đã xảy ra trong thực tế.

Mục đích không chỉ minh họa lý thuyết, mà cho thấy kiến trúc phần mềm là *kết quả của một chuỗi quyết định trong ràng buộc thực tế* — không phải bản thiết kế lý tưởng trên giấy. Mỗi chương trong sách đều liên hệ với ít nhất một khía cạnh của KBM; phụ lục này tập hợp toàn bộ bức tranh để người đọc có thể đối chiếu và rút ra bài học riêng.

1.1. Bối cảnh bài toán

Tại một trường đại học kỹ thuật, các môn học về cơ sở dữ liệu, lập trình mạng, và lập trình thuật toán đòi hỏi sinh viên thực hành với khối lượng lớn bài tập — hàng trăm bài SQL mỗi tuần, hàng chục bài lập trình mạng Java, và các cuộc thi lập trình trực tuyến. Quy trình truyền thống (sinh viên nộp file, giảng viên chấm thủ công) nhanh chóng trở thành nút thắt khi quy mô tăng lên hàng nghìn sinh viên mỗi học kỳ.

1.1.1. Sáu vấn đề cần giải quyết

1. **Chấm thủ công không khả thi.** Giảng viên không thể thực thi và kiểm tra kết quả hàng trăm bài SQL trên nhiều hệ quản trị cơ sở dữ liệu (DBMS) khác nhau trong thời gian hợp lý. Mỗi bài tập cần chạy trên MySQL, PostgreSQL, SQL Server, hoặc Oracle — mỗi hệ có cú pháp và hành vi riêng.
2. **Phản hồi chậm.** Sinh viên thường phải chờ vài ngày mới nhận được kết quả, làm giảm hiệu quả học tập. Vòng lặp *viết — nộp — chờ — sửa* kéo dài thay vì phản hồi tức thì.
3. **Đa dạng CSDL.** Chương trình đào tạo yêu cầu thực hành trên nhiều DBMS. Không có giải pháp chấm tự động nào trên thị trường hỗ trợ đồng thời MySQL, PostgreSQL, SQL Server và Oracle trong cùng một nền tảng — đây là khoảng trống (gap) mà không sản phẩm open-source nào lấp đầy.

4. **Thiếu môi trường sandbox.** Sinh viên cần chạy mã SQL trực tiếp trên cơ sở dữ liệu thật để kiểm tra kết quả, nhưng không thể cho phép truy cập tự do vào hệ thống production — cần cơ chế cô lập (sandbox) ở mức container.
5. **Rủi ro gian lận thi cử.** Các kỳ thi trực tuyến trên phòng máy dễ bị sao chép kết quả giữa các máy nếu không có cơ chế giám sát và khóa môi trường (lockdown) tại máy khách.
6. **Quản lý phân tán.** Thông tin khóa học, bài tập, điểm số, và lịch thi nằm rải rác trên email, bảng tính Excel, và Google Drive — không có hệ thống tập trung, dẫn đến mất đồng bộ và thiếu truy xuất.

1.1.2. Nhóm người dùng

KBM phục vụ ba nhóm chính: **sinh viên** (hàng nghìn mỗi học kỳ — thực hành, thi, nộp bài tập, đồ án), **giảng viên** (vài chục — tạo đề, tổ chức thi, chấm điểm, theo dõi tiến độ), và **quản trị viên** (vài người — quản lý tài khoản, cấu hình hệ thống, giám sát vận hành).

1.1.3. Ràng buộc thực tế

Bối cảnh phát triển KBM chịu những ràng buộc đáng kể — chính các ràng buộc này định hình mọi quyết định kiến trúc:

- **Nhân sự hạn chế:** Đội ngũ core chỉ 1–2 người (tác giả là Product Owner, Principal Architect, và core developer), cộng thêm sinh viên hỗ trợ theo từng đợt đồ án.
- **Hạ tầng giới hạn:** 4 VPS (Virtual Private Server), ngân sách phần cứng hạn chế — không có môi trường cloud hay cluster lớn.
- **Phát triển song song với giảng dạy:** Hệ thống được xây dựng *trong khi vẫn phục vụ giảng dạy thực tế*, không có giai đoạn “đóng cửa để xây lại”.
- **Tích hợp hệ thống cũ:** Phải kết nối với hệ thống quản lý đào tạo (QLĐT) của trường — một hệ thống legacy không có tài liệu API chính thức.

① Ý nghĩa kiến trúc

Những ràng buộc này giải thích tại sao nhiều quyết định kiến trúc trong KBM là *trade-off có chủ đích* (conscious trade-off) thay vì thiết kế lý tưởng. Ví dụ: dùng shared database thay vì database-per-service (xem §E.5) không phải vì thiếu hiểu biết, mà vì team size và deployment topology không cho phép vận hành hàng chục database độc lập. Đây chính là thực tế mà phần lớn sách giáo khoa kiến trúc bỏ qua.

1.2. Bốn trụ cột nghiệp vụ

KBM được tổ chức quanh bốn trụ cột nghiệp vụ chính, mỗi trụ cột tương ứng với một nhóm Bounded Context riêng biệt. Ngoài ra, hệ thống còn hai thành phần hỗ trợ quan trọng.

Trụ cột	Chức năng chính	Yêu cầu kiến trúc đặc trưng
SQL Lab	Chấm SQL tự động trên 4 DBMS, 2 chế độ (Practice đồng bộ + Contest bất đồng bộ), phản hồi AI	Sandbox cô lập mỗi DBMS, xử lý bất đồng bộ qua message broker khi thi
Network Lab	Chấm lập trình mạng Java: 7 nhóm giao thức (TCP → gRPC), 20 cổng	Standalone Java runtime, plugin system, khởi động nhanh
Course Mgmt	Quản lý khóa học, bài tập, MCQ, đồ án, điểm danh, GitHub Classroom	CRUD + tích hợp external (QLĐT, GitHub API)
DevOps Lab	Thực hành K8s/Docker: container-in-container, web terminal, auto-grading	Ngôn ngữ Go, k3s + Sysbox, wildcard DNS

Bảng 1.1: Bốn trụ cột nghiệp vụ KBM và yêu cầu kiến trúc đặc trưng

1.2.1. SQL Lab — Chấm SQL tự động

SQL Lab là trụ cột phức tạp nhất về mặt kiến trúc. Thị trường online judge chỉ có các hệ thống dành cho ngôn ngữ lập trình truyền thống (compile → run → compare output), không có giải pháp tương đương cho SQL — vì SQL không tuân theo mô hình này. KBM giải quyết bằng cách tiếp cận sáng tạo: *thực thi câu SQL trên sandbox CSDL thật, so sánh kết quả qua băm SHA-256 của result set*.

Hệ thống hỗ trợ hai chế độ vận hành:

- **Practice Mode (đồng bộ):** Sinh viên gửi câu SQL → hệ thống thực thi trên sandbox tương ứng (MySQL, PostgreSQL, SQL Server, hoặc Oracle) → so sánh kết quả → trả phản hồi tức thì. Thời gian phản hồi trung bình dưới 2 giây. Tích hợp AI feedback (GPT-4o-mini) để giải thích lỗi logic.
- **Contest Mode (bất đồng bộ qua Kafka):** Trong kỳ thi, hàng trăm sinh viên gửi bài đồng thời. Hệ thống đẩy submission vào Kafka topic → judge worker tiêu thụ (consume) và chấm tuần tự → kết quả đẩy lại qua WebSocket. Thời gian enqueue dưới 50ms, sinh viên không bị block chờ kết quả.

Kết quả chấm (verdict) bao gồm: AC (Accepted), WA (Wrong Answer), TLE (Time Limit Exceeded), RTE (Runtime Error), CE (Compilation Error) — cùng hệ thống phán định với các online judge truyền thống. Cơ chế sandbox sử dụng table prefix riêng cho mỗi DBMS instance, kết hợp topological sort để thiết lập schema đúng thứ tự dependency.

1.2.2. Network Lab — Chấm lập trình mạng

Network Lab phục vụ thực hành lập trình mạng Java, bao gồm 7 nhóm giao thức — từ socket cơ bản đến gọi thủ tục từ xa hiện đại:

1. **TCP** — 6 loại (echo, file transfer, multi-thread, v.v.)
2. **UDP** — 3 loại (datagram, multicast, broadcast)
3. **RMI** — 4 loại (registry lookup, callback, dynamic proxy, security policy)
4. **SOAP** — Web service cổ điển
5. **REST** — HTTP API
6. **gRPC** — Framework RPC hiệu năng cao

Điểm đặc biệt: sinh viên viết chương trình mạng *standalone Java* (không cần thư viện bên ngoài), gửi lên hệ thống. Judge sử dụng cơ chế *dynamic handler reloading* — một dạng plugin system cho phép tải và thực thi mã sinh viên mà không cần khởi động lại judge service. Hệ thống phân bổ 20 cổng mạng (port) riêng biệt cho các nhóm giao thức.

1.2.3. Course Management — Quản lý đào tạo

Trụ cột này bao quát các nghiệp vụ quản lý đào tạo truyền thống, nhưng với tích hợp sâu vào hệ sinh thái KBM:

- **Quản lý khóa học:** Tạo, cấu hình, phân quyền giảng viên.
- **Bài tập:** Hỗ trợ cả bài tập cá nhân và nhóm, định dạng rubric chấm điểm.
- **Thi trắc nghiệm (MCQ):** Ngân hàng câu hỏi, tạo đề ngẫu nhiên, chấm tự động.
- **Đồ án tốt nghiệp:** Quy trình 2 cấp duyệt → bảo vệ → chấm rubric trước hội đồng.
- **Điểm danh:** Session-based, tích hợp lịch học.
- **Grade Scheme và CLO:** Cấu trúc điểm theo component, ánh xạ Chuẩn đầu ra (CLO — Course Learning Outcome).
- **GitHub Classroom:** Đồng bộ assignment, roster linking, theo dõi tiến độ commit.

1.2.4. DevOps Lab — Thực hành DevOps/Kubernetes

DevOps Lab — trụ cột mới nhất — được phát triển bằng Go thay vì Java, một quyết định có chủ đích:

- **Binary nhỏ gọn:** 15 MB (Go binary) so với 200 MB+ (Spring Boot JAR + JRE).
- **Khởi động nhanh:** Mili-giây thay vì vài giây.
- **Goroutine:** Mô hình concurrent hiệu quả cho reverse proxy xử lý nhiều kết nối đồng thời.
- **client-go SDK:** Tương tác native với Kubernetes API.

Kiến trúc sử dụng **mono-repo multi-binary**: một repository Go sinh ra 3 binary riêng biệt (`api` , `router` , `grader`). Hạ tầng dựa trên **k3s** (Kubernetes nhẹ) kết hợp **Sysbox** — runtime cho phép container-in-container an toàn mà không cần quyền privileged. Mỗi sinh viên nhận một *Lab Pod* cô lập với TTL (Time-To-Live) tự động thu hồi tài nguyên.

1.2.5. Thành phần hỗ trợ

Ngoài bốn trụ cột chính, KBM có hai thành phần hỗ trợ quan trọng:

Code Judge (`kbm-judge-code`) — Chấm thuật toán đa ngôn ngữ (C/C++/Java). Thể hiện rõ quyết định *build vs buy*: thể hệ đầu tiên (Gen 1) được phát triển nội bộ nhưng **không có sandbox** — mã sinh viên chạy trực tiếp trên server, tạo rủi ro bảo mật nghiêm trọng. Thể hệ thứ hai (Gen 2) chuyển sang tích hợp **DOMjudge** — hệ thống chấm mã nguồn mở chuẩn ICPC, cung cấp sandbox đầy đủ. Quyết định này minh họa bài học build-vs-buy trong Chương 3 và Chương 10.

Desktop Exam Client (`kbm-exam-client`) — Ứng dụng máy tính để bàn dành cho phòng máy thi, thực hiện *lockdown* môi trường: chặn ứng dụng ngoài, giới hạn IP cho phép (whitelist), và giám sát (proctoring) tại máy khách. Client sử dụng chung JWT token với `kbm-auth` , đảm bảo nhất quán xác thực.

1.3. Quá trình tiến hóa — Từ Monolith đến Microservices

KBM không được thiết kế microservices ngay từ đầu. Hệ thống hiện tại là kết quả của gần một thập kỷ tiến hóa, qua 5 giai đoạn (era) — mỗi giai đoạn bắt đầu bằng một vấn đề thúc đẩy (trigger) và kết thúc bằng một bài học kiến trúc.

1.3.1. Era 1: Distributed Monolith (2016–2022)

Trigger: Nhu cầu ban đầu về chấm bài lập trình và quản lý đào tạo.

Giai đoạn đầu tiên không phải là một monolith duy nhất, mà là *ba hệ thống MVC rời rạc* — ba silo độc lập, mỗi silo là một monolith:

1. **Hệ thống quản lý đào tạo** — Monolith ASP.NET MVC, quản lý khóa học, bài tập, điểm số, tài khoản người dùng.

2. **Code Judge (Gen 1)** — Hệ thống chấm thuật toán C/C++ và Java, tự phát triển. Đặc điểm: *không có sandbox* — mã sinh viên chạy trực tiếp trên server.

3. **Network Judge** — Monolith MVC riêng biệt cho chấm lập trình mạng TCP/UDP/RMI.

Ba hệ thống hoạt động độc lập: không chia sẻ dữ liệu, không chia sẻ danh tính người dùng, không giao tiếp với nhau. Thomas Erl gọi đây là kiến trúc *silos-based* — mỗi ứng dụng là một “ốc đảo” với logic trùng lặp và chi phí bảo trì cao.

Bài học: Distributed monolith khác xa microservices. Ba hệ thống rời rạc không tự động mang lại lợi ích của kiến trúc phân tán — ngược lại, chúng có *nhược điểm của cả monolith (coupling nội bộ) lẫn distributed systems (không nhất quán dữ liệu)* mà không được hưởng ưu điểm của bên nào.

1.3.2. Era 2: SQL Judge — Sáng tạo từ ràng buộc (2023)

Trigger: Nhu cầu chấm SQL tự động — không có sản phẩm nào trên thị trường đáp ứng.

Đây là cột mốc quan trọng nhất trong lịch sử phát triển KBM. Thị trường online judge chỉ có hệ thống cho ngôn ngữ lập trình (compile → run → compare), nhưng SQL không tuân theo mô hình này — không có stdin/stdout, kết quả là *result set* phụ thuộc vào dữ liệu mẫu.

Giải pháp: thực thi câu SQL trên sandbox CSDL thật (mỗi DBMS một container riêng), so sánh result set bằng băm SHA-256. Cơ chế cô lập sử dụng table prefix kết hợp topological sort cho schema dependency. Hệ thống hỗ trợ đồng thời 4 DBMS — điều mà không online judge nào khác làm được tại thời điểm đó.

Bài học: Đổi mới thường sinh ra từ ràng buộc. Khi không có giải pháp có sẵn, việc thiết kế từ đầu (greenfield) buộc phải suy nghĩ sâu về bài toán — và đôi khi tạo ra giải pháp tốt hơn cả những gì thị trường cung cấp.

1.3.3. Era 3: Thiết kế Microservices (2024)

Trigger: Sự phức tạp gia tăng khi tích hợp SQL Judge vào hệ thống hiện có.

Kiến trúc microservices được áp dụng, nhưng không phải dạng *big-bang migration* (viết lại toàn bộ). Thay vào đó, KBM phát triển theo kiểu *organic growth* (tăng trưởng hữu cơ): thiết kế microservices cho SQL Judge mới, rồi dần dần tách các thành phần cũ.

Dấu hiệu của sự phát triển hữu cơ vẫn còn hiện diện: `spring.application.name = "app"` giống nhau giữa `kbm-core` và `kbm-academic` — bằng chứng rằng hai service này từng là một. Shared database `app_db` là trade-off có chủ đích: team nhỏ, cùng deploy cycle, ưu tiên tốc độ phát triển hơn tách biệt hoàn toàn.

Bài học: Microservices trong thực tế hiếm khi bắt đầu “sạch”. Sự tiến hóa hữu cơ để lại dấu vết (architectural debt), và nhận diện được các dấu vết này là bước đầu tiên để cải thiện (xem chi tiết tại §E.5).

1.3.4. Era 4: Migration và Mở rộng (2025)

Trigger: Nhu cầu tách biệt nghiệp vụ đang chồng chéo và mở rộng tính năng mới.

Giai đoạn này đánh dấu loạt thay đổi lớn:

- **Tách service:** `kbm-academic` được tách ra từ `kbm-core`, tuy vẫn dùng chung database (shared DB, giai đoạn chuyển tiếp).
- **Kafka:** Triển khai message broker cho Contest Mode, giải quyết bài toán *rush hour* — hàng trăm submission đồng thời trong phút cuối kỳ thi (xem Sự cố 1, §E.6).
- **Network Judge mở rộng:** TCP/UDP/RMI → thêm SOAP → REST → gRPC, phản ánh thay đổi chương trình đào tạo.
- **Build → Buy:** Code Judge Gen 1 (tự phát triển, không sandbox) → Gen 2 (DOMjudge, open-source chuẩn ICPC).
- **Desktop Exam Client:** Ứng dụng lockdown cho phòng máy thi.
- **OAuth2 mở rộng:** Thêm Microsoft OAuth2, email verification, Cloudflare Turnstile.

- **Observability:** Prometheus metrics, Correlation ID (chưa hoàn chỉnh — xem trade-off tại §E.5).

Bài học: Migration không phải sự kiện một lần (one-time event) mà là quá trình liên tục. Mỗi bước migration phải mang lại giá trị ngay lập tức, không chờ đến khi “migration xong”.

1.3.5. Era 5: Polyglot — DevOps Lab (2025–2026)

Trigger: Tài nguyên VPS hạn chế, Java quá nặng cho workload proxy/grading.

Quyết định sử dụng Go thay vì Java cho DevOps Lab là minh chứng cho nguyên tắc *right tool for the right job*. Với 4 VPS giới hạn, binary Go 15 MB khởi động trong mili-giây là lựa chọn thực dụng hơn Spring Boot JAR 200 MB+.

Kiến trúc mono-repo multi-binary (1 repo → 3 binary: `api`, `router`, `grader`) khác biệt hoàn toàn so với multi-repo microservices của LMS Java. Sự khác biệt này minh họa rằng “microservices” không đồng nghĩa với “mỗi service một repo” — cấu trúc repo phụ thuộc vào ngữ cảnh đội ngũ và ngôn ngữ.

Bài học: Polyglot architecture (đa ngôn ngữ) mang lại lợi thế kỹ thuật, nhưng đòi hỏi đội ngũ phải thành thạo nhiều ngôn ngữ — một chi phí phải được đánh giá rõ ràng (xem trade-off tại §E.5).

🕒 Timeline tổng quan

2016: Distributed Monolith (3 MVC) → **2023:** SQL Judge (cột mốc sáng tạo) → **2024:** Microservices (organic growth) → **2025:** Migration + Kafka + DOMjudge → **2025–2026:** Polyglot Go DevOps Lab. Tổng cộng 10 năm phát triển, microservices chính thức từ 2024.

1.4. Kiến trúc hiện tại

1.4.1. Bounded Contexts

Kiến trúc KBM tổ chức theo 9 Bounded Context (BC), gồm 7 core và 2 supporting:

1. **Identity and Access** (`kbm-auth`) — Quản lý danh tính, RBAC, JWT, OAuth2 (Google/Microsoft), SSO.
2. **SQL Practice** (`kbm-core` + `kbm-judge-sql` + sandbox containers) — Chấm SQL trên 4 DBMS.
3. **Network Practice** (`kbm-judge-network`) — Chấm lập trình mạng 7 nhóm giao thức.
4. **Code Practice** (`kbm-judge-code`) — Chấm thuật toán, tích hợp DOMjudge.
5. **Academic Management** (`kbm-academic`) — Quản lý khóa học, bài tập, MCQ, đồ án, điểm danh.
6. **Examination and Proctoring** (logic trong `kbm-core`, mục tiêu tách thành `kbm-exam`) — Quản lý kỳ thi, contest, anti-cheat, lockdown client.
7. **DevOps Practice** (`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`) — Nền tảng thực hành DevOps bằng Go.
8. **Communication** (`kbm-notification`) — Supporting BC: đẩy thông báo real-time qua WebSocket, tiêu thụ Kafka events.
9. **Platform / Infrastructure** (`kbm-gateway`, `kbm-registry`, `kbm-shared`) — Supporting BC: Gateway, service discovery, thư viện dùng chung (shared library).

 God Service

kbm-core hiện chứa cả SQL Practice (BC #2) lẫn Examination (BC #6) — đây là ví dụ điển hình của anti-pattern *God Service* (xem Phụ lục D). Lưu ý rằng **kbm-core** phát triển hữu cơ từ Era 3, khi ranh giới BC chưa được xác định rõ. Mục tiêu tương lai là tách phần thi cử thành **kbm-exam** riêng biệt.

1.4.2. Tech Stack

KBM là hệ thống **polyglot** — hai nền tảng công nghệ khác nhau cho hai nhóm workload:

Khía cạnh	LMS Main (Java)	DevOps Lab (Go)
Ngôn ngữ	Java 17 + Spring Boot 3.x	Go 1.x + Chi v5
Kiến trúc repo	Multi-repo microservices (10+ repos)	Mono-repo multi-binary (1 repo → 3 binary)
Database	PostgreSQL + MySQL + SQL Server (JPA)	PostgreSQL (sqlx raw SQL)
Auth	Spring Security + JWT	Shared JWT từ kbm-auth (shared secret)
Runtime	Docker Compose	k3s + Sysbox
Migration tool	JPA auto-DDL	golang-migrate
Frontend	2 apps (React + TS + Vite)	1 app (kế thừa stack LMS)

Bảng 1.2: So sánh tech stack giữa LMS Main (Java) và DevOps Lab (Go)

1.4.3. Danh mục dịch vụ

Tên Service Kỹ Thuật	Bounded Context	Mô tả vai trò
kbm-auth	Identity and Access	Quản lý danh tính người dùng, đăng nhập, phân quyền (RBAC), sinh và xác thực JWT token, OAuth2 (Google/Microsoft), SSO.
kbm-core	SQL Practice + Examination	Dịch vụ kbm-core : quản lý cuộc thi, bài tập SQL, quản lý bài nộp, chống gian lận, thống kê. Là dịch vụ ôm đồm (God Object / Monolithic service) tạm thời chứa cả logic thi cử (mục tiêu tương lai sẽ tách rời thành kbm-exam).
kbm-judge-sql	SQL Practice	Điều phối chấm SQL: nhận thông điệp từ Kafka, phân bổ cho các sandbox tương ứng với từng hệ quản trị CSDL, tích hợp tính năng phản hồi tự động bằng trí tuệ nhân tạo (AI feedback).
kbm-sandbox-mysql , kbm-sandbox-postgres , kbm-sandbox-sqlserver , kbm-sandbox-oracle	SQL Practice	Bốn bộ thực thi (executor) chạy mã SQL trong môi trường cô lập trên từng container CSDL – mỗi DBMS một executor riêng.
kbm-judge-network	Network Practice	Chấm điểm lập trình mạng Java (TCP/UDP/RMI/SOAP/REST/gRPC), hỗ trợ môi trường thực thi Java độc lập (standalone Java) và cơ chế tải lại động trình xử lý (dynamic handler reloading).
kbm-judge-code	Code Practice	Chấm điểm thuật toán đa ngôn ngữ (C/C++/Java...). Thế hệ đầu tiên được phát triển nội bộ không có môi trường cô lập (sandbox); thế hệ thứ hai tích hợp DOMjudge để cung cấp môi trường cô lập mã nguồn mở.
kbm-exam-client	Examination and Proctoring	Ứng dụng máy tính để bàn (desktop) dành cho phòng máy: khóa môi trường thi (lockdown), danh sách IP cho phép, giám thị (proctoring) tại máy khách; sử dụng chung JWT của kbm-auth .
kbm-academic	Academic Management	Quản lý khóa học, bài tập dạng rubric, điểm danh, MCQ, syllabus/CLO, GitHub Classroom, đồ án tốt nghiệp.
kbm-notification	Communication	Đẩy thông báo thời gian thực (kết quả chấm bài) qua WebSocket; là bộ tiêu thụ (consumer) của Kafka.
kbm-gateway	Platform / Infra	Gateway duy nhất: định tuyến (path-based), giới hạn lưu lượng (Rate Limiting), xác thực JWT ở tầng biên (edge authentication), cấu hình CORS.

Bảng 1.3: Danh sách dịch vụ KBM theo Bounded Context

 Convention

Xuyên suốt các chương, để đảm bảo tính thực tiễn khi ánh xạ từ sơ đồ kiến trúc xuống mã nguồn hoặc cấu hình triển khai (Docker/Kubernetes), sách sử dụng trực tiếp các định danh kỹ thuật có tiền tố **kbm-*** này. Khi tự xây dựng các hệ thống Microservices, đội ngũ phát triển cũng nên duy trì một danh mục dịch vụ (service catalog) tương tự để tránh tình trạng bất đồng ngôn ngữ (Ubiquitous Language) giữa các nhóm phát triển.

1.4.4. Tích hợp hệ thống bên ngoài

KBM tích hợp với nhiều hệ thống bên ngoài theo ba kiểu khác nhau, mỗi kiểu đặt ra thách thức kiến trúc riêng:

Hệ thống	Kiểu tích hợp	Đặc điểm và Thách thức
QLĐT của trường	Legacy — không có tài liệu API	Phải reverse-engineer API (auth qua Base64, redirect chain, parse token). Cần Anti-Corruption Layer (ACL). Rủi ro cao: API có thể thay đổi không báo trước.
GitHub Classroom	Documented API	REST API chuẩn, OAuth2 App. Tích hợp: sync assignment, roster linking, progress tracking.
Google / Microsoft OAuth2	Standard protocol	OpenID Connect chuẩn. Thách thức: quản lý identity merge khi cùng một người dùng đăng nhập qua nhiều provider (xem Sự cố 4, §E.6).

Bảng 1.4: Ba kiểu tích hợp hệ thống bên ngoài trong KBM

1.4.5. Mô hình triển khai

Trong hạ tầng 4 VPS, hai máy chủ chính đảm nhận các thành phần cốt lõi:

- **VPS1 (Docker Compose):** Chạy toàn bộ Java microservices (10 service), PostgreSQL, MySQL, SQL Server, Kafka, Prometheus, Grafana, Loki. Đây là môi trường Docker Compose đơn máy (single-host) — trade-off về high availability để đổi lấy đơn giản vận hành.
- **VPS2 (k3s cluster):** Chạy DevOps Lab (Go services), reverse proxy với wildcard DNS (`{labId}-{port}.lab.devops.example.com`), Sysbox runtime, và các lab pod của sinh viên. Cluster k3s sử dụng ResourceQuota và LimitRange để giới hạn tài nguyên mỗi lab pod.

1.5. Trade-offs và Gaps — Bài học từ thực tế

Mỗi hệ thống phần mềm thực tế đều chứa trade-off — những quyết định *chấp nhận hạn chế ở khía cạnh này để đạt được lợi ích ở khía cạnh khác*. Với KBM, các trade-off không phải điều xấu hổ cần che giấu, mà là *chất liệu quý giá nhất* cho việc học kiến trúc: chúng ta học được nhiều hơn từ một quyết định “đủ tốt trong bối cảnh” so với một thiết kế lý tưởng trên lý thuyết.

Dưới đây là 25 trade-off được phân loại theo 3 mức nghiêm trọng. Mỗi mục kèm theo chương tham chiếu trong sách — nơi vấn đề được thảo luận chi tiết.

1.5.1. Critical — 8 mục (tính đúng đắn / bảo mật)

Trade-off	Mô tả ngắn	Chương
Thiếu idempotency	Gây duplicate data, ảnh hưởng kết quả thi	Ch.5–6
Internal service không xác thực	Service gọi nhau không qua auth → lỗ hổng bảo mật	Ch.9
Secret management thô sơ	JWT secret dạng plaintext trong config	Ch.9
Token lưu localStorage	XSS → token theft	Ch.9
Judge code không sandbox (Gen 1)	Mã sinh viên chạy trực tiếp trên server	Ch.9–10
CORS allow all origins	Cross-site attack vector	Ch.8
Shared Database giữa service	Schema coupling → cascading failures tiềm ẩn	Ch.7
JWT validate trùng lặp	Mỗi service tự validate → logic drift → inconsistent authorization	Ch.9

Bảng 1.5: Trade-off mức Critical — ảnh hưởng tính đúng đắn và bảo mật

1.5.2. Major — 11 mục (khả năng mở rộng / bảo trì)

Trade-off	Mô tả ngắn	Chương
Shared library coupling	kbm-shared tạo coupling compile-time giữa services	Ch.2–4
Gọi đồng bộ thiếu fault tolerance	Không có circuit breaker, retry chính sách	Ch.4
Thiếu API versioning	API thay đổi có thể phá vỡ consumer	Ch.3
God Service	kbm-core quá nhiều trách nhiệm	Ch.2
Entity trùng lặp giữa service	Cùng khái niệm, khác model → data drift	Ch.2–7
God Class 1400 dòng	Class vi phạm Single Responsibility	Ch.7
Thiếu domain event	Không phát sự kiện khi state thay đổi → coupling ẩn	Ch.5
Thiếu database migration tool	JPA auto-DDL trong production → rủi ro schema drift	Ch.7–10
Kafka single-node	Không có replication → single point of failure	Ch.5
Correlation ID chưa xuyên suốt	Khó truy vết (trace) request qua nhiều service	Ch.11
Docker Compose single-host	Không có HA → downtime khi VPS gặp sự cố	Ch.12

Bảng 1.6: Trade-off mức Major — ảnh hưởng khả năng mở rộng và bảo trì

1.5.3. Minor — 6 mục (cải thiện chất lượng)

Trade-off	Mô tả ngắn	Chương
Thiếu caching layer	Mọi request đều truy vấn DB trực tiếp	Ch.7
Backup chưa tự động	Rủi ro mất dữ liệu khi sự cố	Ch.12
Polyglot complexity	Java + Go → đội ngũ cần thành thạo 2 ngôn ngữ	Ch.1–12
Mono-repo rebuild	Thay đổi 1 binary → build lại cả 3	Ch.2
DB queue thay vì message broker	DevOps Lab dùng DB polling thay Kafka → không replay	Ch.5
Build vs Buy bộ chấm code	Quyết định chuyển từ tự phát triển sang open-source	Ch.3–10

Bảng 1.7: Trade-off mức Minor — cải thiện chất lượng

❗ Tại sao công bố trade-off?

Việc liệt kê 25 trade-off — bao gồm cả những vấn đề nghiêm trọng như “không có sandbox” hay “JWT plaintext” — là có chủ đích. Mục tiêu giáo dục không phải trình bày một hệ thống lý tưởng (điều không tồn tại trong thực tế), mà cho thấy *mỗi quyết định đều có bối cảnh*. Trade-off Shared DB là ví dụ: với team 1–2 người và 4 VPS, việc vận hành 10+ database độc lập là không khả thi — shared DB là lựa chọn thực dụng, kèm theo kế hoạch tách dần khi đủ điều kiện (xem chi tiết trong Chương 7).

1.6. Sự cố thực tế — Bài học vàng

Bốn sự cố dưới đây đã xảy ra trong quá trình vận hành KBM. Mỗi sự cố được trình bày theo format: **Vấn đề** → **Nguyên nhân gốc** → **Cách xử lý** → **Bài học kiến trúc**.

1.6.1. Sự cố 1: Duplicate Submissions trong kỳ thi

Vấn đề. Trong một kỳ thi SQL online, một số bài nộp bị trùng lặp (duplicate) — cùng một sinh viên, cùng một đề bài, cùng nội dung, nhưng xuất hiện hai lần trong hệ thống. Kết quả chấm bị ảnh hưởng.

Nguyên nhân gốc. Thiếu idempotency key trong API nhận bài nộp. Khi mạng timeout, client tự động retry → server nhận cùng một request hai lần và xử lý cả hai như request mới.

Cách xử lý. Thêm idempotency key (UUID) cho tất cả write operation quan trọng. Server kiểm tra key trước khi xử lý: nếu đã tồn tại → trả lại kết quả cũ, không tạo bản ghi mới.

Bài học kiến trúc. Idempotency không phải “nice-to-have” — đây là yêu cầu bắt buộc cho mọi write operation trong hệ thống phân tán. Mạng sẽ timeout, client sẽ retry. Xem chi tiết tại Chương 5 (Idempotent Consumer) và Chương 6 (Saga Pattern).

1.6.2. Sự cố 2: Dữ liệu không nhất quán giữa các bảng

Vấn đề. Hai service (`kbm-core` và `kbm-academic`) sử dụng chung database nhưng tạo các bảng trùng lặp cho cùng một khái niệm nghiệp vụ. Ví dụ: `class_room` và `as_course` cùng lưu thông tin khóa học. Dữ liệu giữa hai nhóm bảng dần trôi dạt (drift) — cập nhật ở bảng này không phản ánh ở bảng kia, dẫn đến sinh viên thấy thông tin khóa học khác nhau tùy theo tính năng đang sử dụng.

Nguyên nhân gốc. Hai giai đoạn phát triển (thực tế là cùng tác giả nhưng ở hai thời điểm khác nhau) xây dựng entity mapping độc lập, tạo ra 9 điểm xung đột: 3 cặp bảng trùng lặp, 3 bảng cùng tên nhưng mapping khác, 3 bảng bị ghi từ nhiều service.

Cách xử lý. Thực hiện migration 3 pha: (1) Chuẩn hóa entity dùng chung qua `@Immutable` + `ReadOnlyRepository` trong `kbm-shared`; (2) Merge bảng trùng lặp (`class_room` + `as_course` → `sys_course`); (3) Thiết lập quy ước ownership: tiền tố `sys_` = shared, `as_` = assignment domain, một writer duy nhất cho mỗi khái niệm.

Bài học kiến trúc. Shared database không tự động gây vấn đề — vấn đề nằm ở *thiếu quy ước ownership* rõ ràng. Khi nhiều service cùng ghi vào cùng một khái niệm mà không có “nguồn sự thật duy nhất” (single source of truth), data drift là hệ quả tất yếu. Xem chi tiết tại Chương 7 (Quản lý Dữ liệu) và Chương 2 (Bounded Context).

1.6.3. Sự cố 3: Multi-instance Judge — Định tuyến sai

Vấn đề. Khi scale judge service lên nhiều instance, giao thức chấm 2 pha gặp lỗi: Bước 1 (nhận đề bài, sinh request ID) được xử lý bởi Instance A; Bước 2 (gửi bài giải kèm request ID) được chuyển đến Instance B. Instance B không biết request ID từ Instance A → từ chối bài nộp.

Nguyên nhân gốc. Giao thức chấm *stateful* (lưu request ID trong memory của instance) nhưng hạ tầng scale *stateless* (load balancer phân bổ ngẫu nhiên). Hai mô hình xung đột nhau.

Cách xử lý. Ba hướng được cân nhắc: session affinity (sticky session) tại load balancer, shared state qua Redis hoặc database, hoặc thiết kế lại giao thức thành *stateless* hoàn toàn. Đội ngũ chọn phương án giải quyết tận gốc — **thiết kế lại giao thức thành stateless**: bước gửi bài giải được sửa để tự mang đủ ngữ cảnh chấm (self-contained), nhờ đó bất kỳ instance nào cũng xử lý được yêu cầu mà không cần nhớ request ID từ bước trước. Hai phương án còn lại chỉ che triệu chứng ở tầng hạ tầng; *stateless redesign* loại bỏ hẳn lớp lỗi và khớp với nguyên tắc thiết kế ở Chương 4.

Bài học kiến trúc. Stateful protocol + stateless scaling = xung đột. Khi thiết kế giao thức giao tiếp giữa các service, cần *giả định rằng bất kỳ instance nào cũng có thể xử lý bất kỳ request nào* — đây là nguyên tắc *stateless design* (Chương 4). Nếu buộc phải giữ state, cần externalize state ra shared store (Chương 12).

1.6.4. Sự cố 4: Identity Merge — Khủng hoảng danh tính

Vấn đề. Khi mở rộng OAuth2 (thêm Microsoft bên cạnh Google), phát hiện nhiều người dùng có 2–3 tài khoản riêng biệt cho cùng một con người — đăng nhập qua Google tạo tài khoản A, qua Microsoft tạo tài khoản B, qua email/password tạo tài khoản C. Dữ liệu học tập (điểm, bài nộp, lịch sử) bị phân tán qua nhiều danh tính.

Nguyên nhân gốc. Thiếu cơ chế *identity linking* — hệ thống tạo tài khoản mới cho mỗi provider mà không kiểm tra email trùng. Đây là vấn đề phổ biến khi thêm OAuth2 provider vào hệ thống đã có user base.

Cách xử lý. Xây dựng cơ chế identity merge: liên kết các tài khoản cùng email thành một danh tính duy nhất, di chuyển (migrate) toàn bộ dữ liệu liên quan sang identity chính. Quá trình gộp chạy tự động theo email đã xác thực — các khóa ngoại được trỏ lại về identity chính — kèm bước đối soát thủ công cho các trường hợp biên. Kết quả: toàn bộ dữ liệu học tập (điểm, bài nộp, lịch sử) được bảo toàn, không mất dữ liệu.

Bài học kiến trúc. Identity management trong hệ thống multi-provider cần được thiết kế từ đầu với khái niệm *identity* tách biệt khỏi *credential*. Một người dùng = một identity, có thể liên kết nhiều credential (email, Google, Microsoft). Xem chi tiết tại Chương 2 (BC discovery cho Identity and Access) và Chương 9 (Bảo mật).

1.7. Ánh xạ Case Study ↔ Chương sách

Mỗi chương trong sách đều tham chiếu đến ít nhất một khía cạnh của KBM. Bảng dưới đây tổng hợp ánh xạ giữa nghiệp vụ, chương, pattern/bài học, và sự cố liên quan (nếu có).

Chương	Nghịệp vụ KBM liên quan	Pattern / Bài học chính	Sự cố
Ch.1	Tổng quan hệ thống, từ các monolith rời rạc → microservices	Monolith Hell, Fast Flow, Quality Attributes	—
Ch.2	7+2 BC discovery, god service, entity duplication	DDD, Bounded Context, Context Map	#4
Ch.3	Multi-protocol judge, DOMjudge ACL, exam-client multi-client	API Design, Build vs Buy, Versioning	—
Ch.4	SqlExecutorService, OpenFeign, QLDT integration, fault tolerance	Sync Communication, Circuit Breaker, ACL	#3
Ch.5	Kafka Contest Mode, DB queue DevOps Lab, idempotency	Async Messaging, Idempotent Consumer	#1, #3
Ch.6	Submit-Judge implicit saga, duplicate submissions	Saga Pattern, Compensating Transactions	#1
Ch.7	Shared DB, JPA auto-DDL vs golang-migrate, data drift	Database per Service, Shared DB trade-off	#2
Ch.8	Spring Cloud Gateway + Go hostname-based routing, CORS	API Gateway, Rate Limiting, Edge Auth	—
Ch.9	SSO cross-platform, OAuth2, no-sandbox risk, exam lockdown	Security, JWT, OAuth2, Secret Management	#4
Ch.10	Migration roadmap, strangler fig, build→buy DOMjudge	Monolith → Microservices, Strangler Fig	—
Ch.11	Correlation ID, Prometheus, Promtail→Loki→Grafana	Observability, Distributed Tracing	—
Ch.12	Docker Compose vs k3s, CI/CD, RBAC, DevOps Lab infra	Deployment, Container Orchestration, IaC	—

Bảng 1.8: Ảnh xạ nghiệp vụ KBM ↔ Chương sách ↔ Sự cố thực tế

1.8. Contributors

Bảng dưới đây ghi nhận các cộng tác viên đã đóng góp trực tiếp vào quá trình phát triển và vận hành hệ thống KBM — case study chính xuyên suốt cuốn sách. Vai trò tổng thể của tác giả (Product Owner, Principal Architect, Technical Lead, Core Developer) được mô tả trong Lời cảm ơn.

Tên	Khóa	Phân hệ / Vai trò
Nguyễn Thành Phước	D21	Judge Service, SQL Lab — đóng góp giai đoạn đầu
Nguyễn Văn Đức	D21	Assignment Service, module thi trắc nghiệm
Nguyễn Trung Hiếu	D21	Assignment Service, module thi trắc nghiệm
Trần Tất Quốc Huy	D17	Triển khai DevOps, hỗ trợ vận hành
Vương Đức Trọng	—	DevOps Lab
Lê Đức Huy	—	DevOps Lab

Bảng 1.9: Danh sách Contributors của KBM